Smart Pharmacy Inventory and Prescription Management System

Course Name:Software Engineering

Course Code:CSA1004

Guided by:
Dr.Anitadavamani K

Presented by:
K.Siva
192525461

Smart Pharmacy Inventory and Prescription Management System

1 Understanding the problem

This report proposes a small pharmacy management platform that helps pharmacy staff keep track of medicine stock, batch numbers, expiry dates, prescriptions, suppliers, and sales. A **batch** is a group of units of one medicine made and supplied together, sharing one batch number and expiry date. The system records and organises this information and raises alerts, while the pharmacist makes every professional decision. The report describes a proposed design, not a completed implementation or measured field study.

Problem and practical example

Pharmacies hold hundreds of medicines, often with several batches of the same medicine on the shelf at once. Each batch has its own quantity and expiry date. When stock is counted by hand or noted on paper, a batch may expire unnoticed at the back of a shelf, a fast-moving medicine may run out before it is reordered, and a prescription-only medicine may be sold without the prescription being recorded. Each of these causes waste, lost sales, or operational errors.

Consider a small pharmacy that stocks an antibiotic in two batches. Batch A expires in six weeks and Batch B in fourteen months. The assistant normally picks whichever box is easiest to reach. The platform lists Batch A as a near-expiry item and suggests dispensing it first, so that the older stock is used before it expires. When the total quantity of the antibiotic falls below its reorder level, a low-stock alert names the medicine and the supplier it is normally ordered from.

Objectives

- Maintain medicine inventory with quantities, units, and storage details.
- Track batch numbers and expiry dates for every stock item.
- Manage prescriptions linked to patients, prescribers, and dispensed medicines.
- Generate low-stock alerts when quantity falls to a configured reorder level.
- Manage supplier information and purchase orders.
- Track medicine transactions: purchases, sales, returns, and write-offs.
- Identify near-expiry and expired products.
- Generate inventory reports for chosen periods.

Scope and boundaries

The first version covers one pharmacy with registered medicines, batches, suppliers, users, prescriptions, a web interface, rule-based alerts, and downloadable reports. A student team can first build the stock and batch modules and then add prescriptions, sales, and reports. Online ordering, insurance claims, e-prescription exchange with hospitals, automatic drug-interaction checking, and fully automatic reordering are outside the initial scope.

The platform records prescriptions and flags medicines that require one; it does not decide whether a prescription is clinically appropriate. That judgment stays with the licensed pharmacist. Patient details are kept to what dispensing needs and must be protected as private records.

People involved and their challenges

Stakeholder	Role and responsibilities	Main challenges
Pharmacist	Checks prescriptions, dispenses medicines, and makes final professional decisions.	Time pressure and the need for accurate stock and batch information.
Pharmacy assistants	Receive stock, process sales, and shelve medicines.	Entering data correctly and quickly during busy periods.
Owner or manager	Approves purchases, reviews reports, and controls costs.	Expired stock losses and uncertain demand.
Suppliers	Deliver medicines with batch and expiry details.	Late deliveries and mismatched orders.
Prescribers (doctors)	Issue prescriptions that the pharmacy records and fills.	Unclear handwriting and incomplete details on paper.
Patients	Receive medicines and counselling.	Waiting time and medicine availability.
System administrators	Manage accounts, backups, and system availability.	Protecting private records and avoiding downtime.
Developers and testers	Build modules, verify behaviour, and fix defects.	Changing requirements and hidden data errors.

Staff need messages such as “Batch B1204 of this medicine expires in 30 days” rather than unexplained codes. The screen must show clearly which stock figures are current and when they last changed. Stock counts must be checked physically from time to time, because a system count can differ from the shelf after breakage, theft, or unrecorded returns.

Existing process and its strengths

Many small pharmacies use paper registers, spreadsheets, or simple billing software. Staff check shelves by sight, remember which medicines sell quickly, and order from familiar suppliers. This process has a low initial cost and uses the experience of the pharmacist. Direct shelf checks can also reveal damaged packaging, wrong storage, and other problems that a database cannot see.

Limitations and gaps to address

Paper registers are slow to search, may omit batch numbers or expiry dates, and make it hard to see which medicines will expire soon. Manual counts give occasional snapshots, so shortages are discovered only when a customer asks. Prescription details written in a notebook can be incomplete, making later checks difficult. Reports for the owner take time to prepare and may contain errors.

The proposed platform addresses these gaps through batch-level records, timestamped transactions, automatic alerts, and quick reports. It helps staff decide what to check, reorder, or sell first. Pharmacist judgment remains necessary because a data-entry error, an unrecorded return, or an unusual demand pattern can make an alert unsuitable.

Architecture and prototype components

The information flow is: pharmacy user → web interface → Flask backend → database → rules engine → alerts and reports. A staff member enters or scans an item; the backend validates the data and stores it; the rules engine checks quantities and expiry dates after each change; and the results appear on the dashboard and in reports.

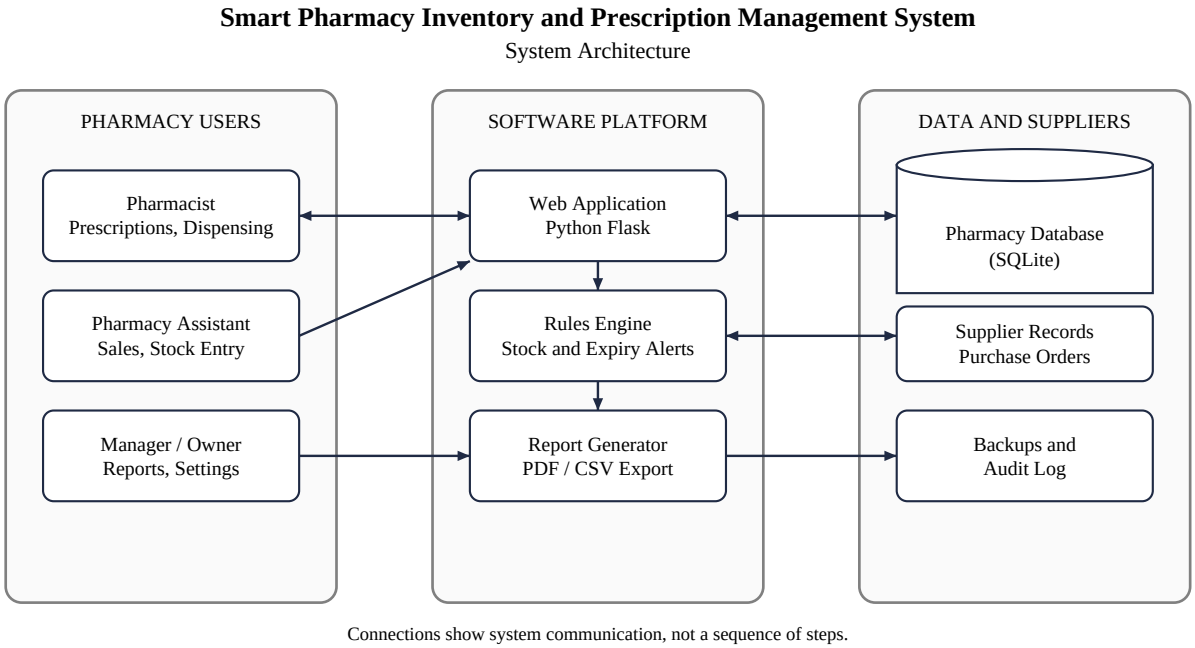


Fig.no:1

The **web interface** is the set of pages used by staff on a computer, tablet, or phone. The **Python Flask backend** is the server program that receives requests, checks data, and controls access. **SQLite** stores records in a single database file. The **rules engine** applies simple, configurable rules for low stock and expiry, and the report generator produces downloadable summaries. An optional barcode scanner can speed up stock entry and sales by reading the medicine code.

This combination suits a small prototype because Flask needs little setup, SQLite avoids a separate database server, and a basic mobile-friendly web page needs no dedicated application. The design keeps each part separate so it can be replaced later.

Six functional modules

Medicine and batch management stores medicine names, categories, units, reorder levels, batch numbers, expiry dates, and quantities. **User management** stores staff accounts and roles. **Prescription management** records patient details, prescriber, date, and prescribed items, and links each dispensed medicine to its prescription. **Sales and transaction tracking** records purchases, sales, returns, and write-offs, and updates stock after each one.

Supplier management stores supplier contacts, supplied medicines, and purchase orders. **Alerts and reports** raises low-stock, near-expiry, and expired-batch alerts and exports inventory, sales, and expiry reports for a chosen period.

Functional requirements

- Provide login and allow authorised users to register medicines, batches, suppliers, and users.
- Store each batch with its number, expiry date, quantity, supplier, and purchase date.

- Record prescriptions and require a linked prescription before dispensing prescription-only medicines.
- Record every transaction with date, time, user, quantity, and batch, and update stock automatically.
- Suggest the earliest-expiring valid batch when dispensing, and block the sale of expired batches.
- Allow authorised users to configure reorder levels and the near-expiry warning period.
- Show low-stock and near-expiry alerts that name the medicine, batch, and suggested action.
- Maintain supplier details and purchase orders linked to received stock.
- Export inventory, sales, and expiry reports for a chosen period.

Illustrative alert rules

For illustration only, suppose a medicine has a reorder level of 50 units. When the total valid quantity across its batches falls to 50 or below, the platform raises a low-stock alert and shows the usual supplier. Suppose also that a batch is flagged as near-expiry when its expiry date is within 90 days. The platform lists it first for dispensing, approach known as **first-expiry-first-out (FEFO)**. A batch past its expiry date is marked expired, excluded from available stock, and blocked from sale.

These values are not universal. The pharmacist or manager should set reorder levels and warning periods for each medicine according to demand, supplier lead time, and storage needs. The prototype recommends a reorder or review; it does not place orders automatically or decide the exact order quantity.

Non-functional requirements

Modularity separates inventory, prescriptions, sales, alerts, and reports so a rule can change without rewriting other code. **Scalability** means supporting growth: every record carries pharmacy, medicine, and batch identifiers, and if the number of users or branches outgrows the prototype, the team can replace SQLite with a server database such as PostgreSQL.

Maintainability means making changes safely through readable code, setup instructions, configuration files, and Git version control, which records changes. **Reliability** requires validating entries, rejecting negative or impossible quantities, preventing duplicate transactions, keeping a transaction history, and backing up records regularly. **Usability** requires simple screens, clear units, readable alerts, and quick search by medicine name or barcode.

Security and privacy require securely hashed passwords, role-based permissions, encrypted HTTPS connections, and an audit log of who changed what. Only authorised pharmacists may approve prescription-only sales or edit prescription records, and only managers may change thresholds. Patient and prescription data must be limited to what is needed and protected from unauthorised viewing.

Proposed acceptance targets, not achieved results: after a sale or stock change is saved, update the stock quantity immediately and show any triggered alert within 60 seconds; search a medicine and display its batches within 3 seconds for a prototype-sized catalogue; and store exactly one record per submitted transaction even if a form is submitted twice. These targets must be verified in prototype testing.

Agile with lightweight Scrum

The Software Development Life Cycle (SDLC) covers requirements, design, development, testing, deployment, and maintenance. Agile performs this work in small stages and uses feedback to adjust priorities. Scrum organises Agile work into short cycles called sprints. A product backlog is an ordered list of needed features and fixes. Four proposed one-week sprints are:

Sprint	Planned work
1	Requirements, interface design, user login, and medicine and supplier registration.
2	Batch and expiry tracking, stock entry, and data validation.
3	Prescriptions, sales transactions, and low-stock and near-expiry alerts.
4	Reports, testing, user feedback, and improvements.

This approach suits changing pharmacy workflows and staff feedback. After each sprint, the team demonstrates working features, reviews feedback, tests changes, updates documentation, and revises the backlog. A fixed sequential Waterfall approach is less flexible here because late discoveries, such as a missing field on the prescription form, may force earlier design stages to be revisited.

Meaningful tests

- Enter a negative quantity, a missing batch number, or an expiry date in the past: reject or flag the entry without changing stock.
- Sell a medicine in quantities that exceed available stock: block the sale and keep stock unchanged.
- Try to sell an expired batch: block the sale and show the reason.
- Dispense a prescription-only medicine without a linked prescription: deny the transaction.
- Reduce stock to the reorder level: raise the low-stock alert once, and not before the level is reached.
- Submit the same sale twice: store one transaction only.
- Attempt a threshold change as an assistant: deny access.
- Compare an exported report's totals with stored records: require an exact match.

Expected outcomes and evaluation

These benefits are expected possibilities, not proven outcomes. **Reduced medicine wastage** follows from earlier warnings about near-expiry batches and FEFO dispensing. **Improved inventory accuracy** follows from recording every purchase, sale, and return with its batch. **Fewer stock shortages** follow from reorder alerts raised before stock runs out, and **better expiry management** from a clear list of batches approaching expiry.

Faster pharmacy operations may result from quick search, barcode entry, and automatic stock updates in place of manual register work. **Improved record management** follows from searchable, timestamped prescription, supplier, and transaction records. The system cannot guarantee savings, since demand, supplier reliability, and staff practice also influence results.

A field evaluation could compare the value of expired stock written off, the number of stock-outs, and the time taken to serve a prescription before and after use, over similar periods. Stock accuracy can be measured by comparing system counts with physical counts. Alert delivery time can be measured from a saved transaction to a displayed alert, and staff feedback can assess whether messages are understandable and useful.

Risks and practical mitigations

Risk or limitation	Practical mitigation
Data-entry errors such as wrong batch, quantity, or expiry date	Validate entries, support barcode scanning, and require confirmation for stock adjustments.
System count differs from physical stock	Carry out regular stock checks and record adjustments with a reason.

Risk or limitation	Practical mitigation
Misuse or exposure of patient and prescription data	Use role-based access, hashed passwords, HTTPS, an audit log, and minimal patient data.
Power or internet failure	Run the prototype locally, back up the database regularly, and keep a paper fallback.
Unsuitable reorder levels and warning periods	Set values per medicine with pharmacist input and review repeated false alerts.
Staff resistance or limited training	Keep screens simple, provide a short guide, and collect feedback each sprint.

The platform records and alerts; it does not give clinical advice. An alert should state the stock or expiry condition and suggest a check, not replace the pharmacist's judgment. Missing or invalid data should lower confidence in an alert instead of being treated as proof that stock is correct.

Future enhancements

Later versions could add automatic purchase-order suggestions based on sales history, barcode and QR-code scanning of every pack, SMS or email alerts, and multi-branch support with a server database. Drug-interaction checking, e-prescription exchange, and insurance billing would need reliable external data sources, legal review, and separate validation. Automatic reordering should be introduced only with manager approval and a safe response to missing data.

Conclusion

The proposed platform gives pharmacy staff accurate batch-level stock records, timely low-stock and near-expiry alerts, organised prescription and supplier records, and quick reports. Keeping the first version to one pharmacy, a web interface, and simple rules makes it achievable for a student team. Its usefulness depends on accurate data entry and the professional judgment of the pharmacist, which the design supports throughout.